Table of Content

ABSTRACT

In the evolving domain of AI-generated text classification, ensuring the reliability of distinguishing human-written content from AI-generated text is becoming increasingly important. This project explores multiple machine learning algorithms including Naive Bayes, Decision Tree, Logistic Regression, Random Forests, SVM, and Deep learning LSTM-based techniques to classify text as either human- or AI-generated. With TF-IDF and tokenization as preprocessing strategies, our system leverages historical text data to train classifiers that predict with high accuracy.To leverage sequential patterns in text, an **LSTM (Long Short-Term Memory)** model is constructed using TensorFlow/Keras with embedding and dropout layers, resulting in superior performance with **99% accuracy**.Comprehensive evaluation through confusion matrices and **ROC-AUC analysis** confirms the LSTM model's robustness. The results underscore that while classical models provide strong baselines, deep learning—especially LSTM—excellently captures contextual nuances required for this classification task.

Introduction

In the digital era, the ability of artificial intelligence (AI) to generate human-like text has seen rapid advancements, especially with the emergence of large language models (LLMs) such as ChatGPT, GPT-4, and others. These models can produce highly coherent and contextually relevant content, making it increasingly difficult to distinguish between AI-generated and human-authored text. This growing overlap presents significant challenges in areas such as academic integrity, misinformation detection, content moderation, and authorship verification.

To address this challenge, text classification techniques that can accurately differentiate between human and AI-generated content have become an essential area of research. Traditional machine learning models—such as Naive Bayes, Logistic Regression, and Random Forest—offer efficient solutions using manually crafted features like term frequency-inverse document frequency (TF-IDF). However, these models may struggle with capturing deeper contextual and semantic patterns in language.

On the other hand, deep learning approaches, especially Recurrent Neural Networks (RNNs) like Long Short-Term Memory (LSTM), excel in modeling sequential data and understanding long-range dependencies in text. These models can learn complex linguistic patterns directly from raw text inputs, making them well-suited for nuanced classification tasks.

In this project, we build and compare multiple classification models to perform binary classification of text into two classes: human-written and AI-generated. We use a labeled dataset, apply text preprocessing, feature extraction, and train several traditional and deep learning models. The project includes evaluation using metrics like accuracy, precision, recall, F1-score, and ROC-AUC to identify the most effective approach for this classification task.

This study not only contributes to the understanding of linguistic differences between AI and human-generated content but also helps build reliable systems for detecting synthetic content in practical applications.

2. Literature Survey

The field of text classification has evolved significantly, particularly with the rise of natural language processing (NLP) and deep learning. This section provides an overview of related work and existing techniques used in distinguishing human-generated text from machine-generated text.

- ## Machine Learning Approaches

  Earlier research in text classification primarily relied on machine learning algorithms. Models such as Naive Bayes, Support Vector Machines (SVM), and Logistic Regression have been widely used due to their simplicity and effectiveness. These models typically use TF-IDF [1, 2] or bag-of-words (BoW) representations, which convert text into numerical features based on word frequency and importance.Early studies used **text complexity features**, such as syllable count, sentence structure, perplexity, and n-grams, combined with machine learning models like SVM, Naive Bayes, and Random Forest — reaching up to **93% accuracy** but I achieved better from SVM and Logistic Regreesion upto 96%.

- ## Deep Learning

  With the development of deep learning, models like Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs) became more popular in NLP tasks. In particular, Long Short-Term Memory (LSTM) networks [3] are effective for modeling sequential data and understanding long-term dependencies.Recurrent neural networks like **LSTM** and **Bi-LSTM** demonstrated strong performance. For instance, Bi-LSTM achieved up to **98.8% F1-score** on English datasets but I only use the LSTM model and achieved accuracy upto 99%.

- ## Challenges

  Vocabulary Overlap: AI models are trained on human-generated content, making their outputs linguistically similar to real human writing.
  Context Sensitivity: Traditional models often lack the ability to understand the semantic flow and coherence of text.
  Evolving Generative Models: As AI models improve, detection becomes more difficult due to increasing realism and diversity of text.

3. Problem Statement

- Despite rapid advances in AI-generated text (e.g., ChatGPT, GPT-4), distinguishing between human-written and AI-generated content remains a significant challenge. Existing manual detection methods are:
  - Time-consuming and *error-prone*, especially at scale
  - Often based on surface-level cues (e.g., style or grammar), which AI models can easily mimic over time
  - Limited in their ability to generalize across different domains or writing styles

- **Objective**

  The core objective of this project is to design and implement a binary classification system that can accurately distinguish between human-generated and AI-generated text using both traditional machine learning algorithms and a deep learning approach (LSTM).

4. Solution Strategy

To effectively distinguish between AI-generated and human-generated text, a hybrid approach that combines traditional machine learning models and deep learning methods is used.

- **Data Preparation**
  - Load data from parquet format
  - Clean and filter samples
  - Map sources into binary classes

- **Feature Extraction**
  - TF-IDF for traditional ML models
  - Tokenization and Padding for LSTM

- **Model Building**
  - Machine Learning Models: NB, SVM, Logistic Regression, RF, DT, KNN
  - Deep Learning Model: LSTM with embedding and dropout

- **Evaluation**
  - Accuracy, Precision, Recall, F1-score
  - Confusion Matrix, ROC-AUC

- **Tools Used**
  - pandas, numpy, scikit-learn, keras, matplotlib

6. Dataset Description

The dataset is a large-scale collection intended for AI Text Detection tasks, particularly for long-form essays. It includes samples from both human-written and AI-generated texts.

- **Source Composition**
  - Human-written text: essays, articles, forum responses
  - AI-generated text: created by GPT-2, GPT-3, GPT-J, and ChatGPT
  - *source: ai 165445 ,human 33486*

- **Format**
  - Stored in Parquet format with text and source columns
  - Dataset containing human and AI-generated text samples is taken from Hugging face (https://huggingface.co/datasets/artem9k/ai-text-detection-pile/tree/main/data)

- **Preprocessing**
  - Mapped GPT variants to a single 'ai' label
  - Binary classification: 0 = human, 1 = ai

- **Challenges**
  - Semantic similarity between human and AI
  - Long-form content handling
  - Class imbalance

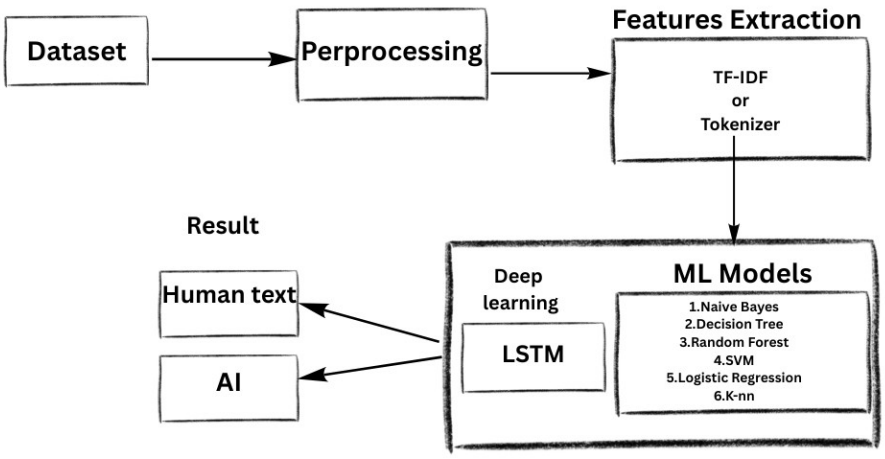
- **Significance**

  This dataset serves as a real-world benchmark for detecting synthetic content and training robust AI detectors.

7. Methodology

This section describes the step-by-step methodology followed to build and evaluate the classification system. It includes preprocessing, model training, and evaluation techniques for both traditional machine learning and deep learning models.

This is the framework of our proposed model (as shown in the below figure) to classify the AI and Human text detection.



1.Data Preprocessing

Preprocessing is critical to ensuring consistent and high-quality inputs for machine learning models. The following steps were applied:

- **Label Filtering**: Only human, gpt2, gpt3, chatgpt, and gptj samples were retained. AI-generated classes were grouped under a single label ai.
- **Null Handling**: Missing or null text entries were removed or replaced with empty strings.
- **Lowercasing**: All text was converted to lowercase for normalization.

- **Label Encoding**:
  - human → 0
  - ai → 1

1. Feature Extraction

Two methods were used based on model requirements:

a. TF-IDF Vectorization

- Used for traditional ML models.
- Transformed raw text into sparse numerical features.
- Configured with:
  - max_features = 2000
  - stop_words = 'english'

b. Keras Tokenizer + Padding

- Used for LSTM model.
- Converted text to sequences of integers.
- Applied padding to ensure fixed-length input (e.g., 200 tokens per sample).

2. Model Training

a. Machine Learning Models

Trained using the scikit-learn library:

- **Naive Bayes**
- **Logistic Regression**
- **Random Forest**
- **Decision Tree**
- **Support Vector Machine (SVM)**
- **K-Nearest Neighbors (KNN)**

Each model was trained using TF-IDF features with an 80-20 train-test split.

b. Deep Learning Model: LSTM

Built using Keras:

- **Embedding Layer**: Converts token IDs to dense vectors.
- **LSTM Layer**: Captures sequential patterns in text.

- **Dropout Layer**: Prevents overfitting.
- **Dense Output Layer**: Sigmoid activation for binary classification.

c. **Hyperparameters:**

- Epochs: 5
- Batch size: 64
- Optimizer: Adam
- Loss: Binary crossentropy

3. Model Evaluation

Each model was evaluated on:

- **Accuracy**: Measures the overall correctness of the model.
- **Precision**: The proportion of positive identifications that were actually correct.
- **Recall**: The proportion of actual positives that were correctly identified.
- **F1-Score: Harmonic mean of precision and recall.**

Visualizations were created to compare performance:

- **Confusion Matrix**: Visual insight into true/false predictions for each class.
- **ROC Curves** for probabilistic models.

8.Results

1. Comparative Analysis

- **Best Accuracy**: Logistic Regression and SVM (~96%)
- **Best Recall for AI Detection**: Random Forest (1.00), KNN (1.00), Decision Tree (0.98)
- **Best Balanced Performance**: LSTM

Table for the performance comparision

Model	Accuracy	Precision (0) / (1)	Recall (0) / (1)	F1-Score (0) / (1)
Naive Bayes	0.942	0.82 / 0.97	0.84 / 0.96	0.83 / 0.96
Random Forest	0.909	0.99 / 0.90	0.46 / 1.00	0.63 / 0.95
Decision Tree	0.923	0.87 / 0.93	0.64 / 0.98	0.73 / 0.95
SVM	**0.964**	**0.90 / 0.98**	**0.88 / 0.98**	**0.89 / 0.98**
K-NN	0.834	0.78 / 0.83	0.02 / 1.00	0.03 / 0.91
Logistic Regression	**0.964**	**0.91 / 0.97**	**0.87 / 0.98**	**0.89 / 0.98**
LSTM	**0.990**	**0.95 / 0.99**	**0.97 / 0.99**	**0.96 / 0.99**

2. Final Model Selection

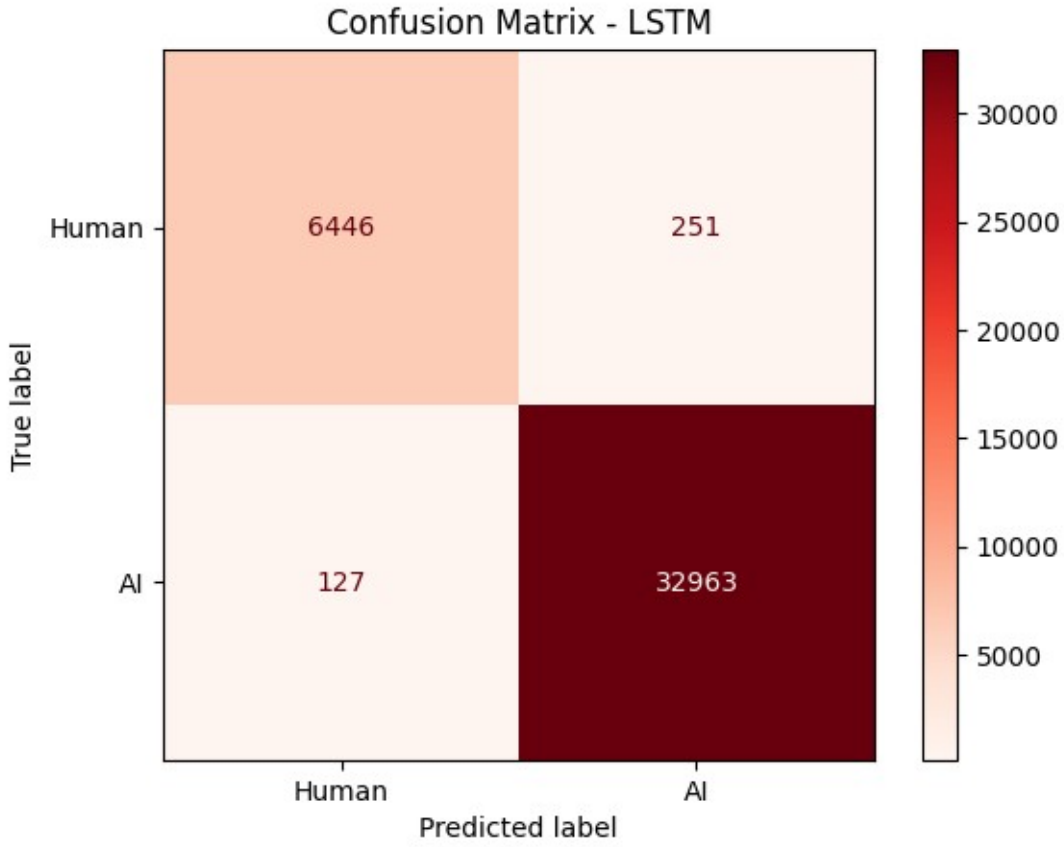
- **Machine Learning Winner**: Logistic Regression (high accuracy, AUC, balance).

- **Best Overall Model**: **LSTM**, due to strong generalization and low misclassification (validated by confusion matrix).

3. Deep Learning (LSTM) Model Performance

Confusion Matrix

- This indicates a **very strong performance** in identifying AI-generated content, with low false positives and false negatives.



9. Implementation

1. Performance of Machine Learning Models

```python
from sklearn.metrics import accuracy_score, classification_report

models = {'Naive Bayes': nb}

for name, model in models.items():
    y_pred = model.predict(X_test_vec)
    print(f"---{name}---")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print(classification_report(y_test, y_pred))
```

```
---Naive Bayes---
Accuracy: 0.9415638273808028
              precision    recall  f1-score   support

           0       0.82      0.84      0.83      6697
           1       0.97      0.96      0.96     33090

    accuracy                           0.94     39787
   macro avg       0.89      0.90      0.90     39787
weighted avg       0.94      0.94      0.94     39787
```

Naive Bayes

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Initialize the model
rf = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42)

# Train the model
rf.fit(X_train_vec, y_train)

# Predict on the test set
y_pred_rf = rf.predict(X_test_vec)

# Evaluate the model
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 0.9091411767662805
              precision    recall  f1-score   support

           0       0.99      0.46      0.63      6697
           1       0.90      1.00      0.95     33090

    accuracy                           0.91     39787
   macro avg       0.95      0.73      0.79     39787
weighted avg       0.92      0.91      0.90     39787
```

Random Forest

```python
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(max_depth=10, random_state=42)
dt.fit(X_train_vec, y_train)

y_pred_dt = dt.predict(X_test_vec)

print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))
```

```
Decision Tree Accuracy: 0.9225375122527458
              precision    recall  f1-score   support

           0       0.87      0.64      0.73      6697
           1       0.93      0.98      0.95     33090

    accuracy                           0.92     39787
   macro avg       0.90      0.81      0.84     39787
weighted avg       0.92      0.92      0.92     39787
```

Decision Tree

```python
from sklearn.svm import LinearSVC

svm = LinearSVC()
svm.fit(X_train_vec, y_train)

y_pred_svm = svm.predict(X_test_vec)

print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
print(classification_report(y_test, y_pred_svm))
```

```
SVM Accuracy: 0.9644104858370824
              precision    recall  f1-score   support

           0       0.90      0.88      0.89      6697
           1       0.98      0.98      0.98     33090

    accuracy                           0.96     39787
   macro avg       0.94      0.93      0.94     39787
weighted avg       0.96      0.96      0.96     39787
```

Linear SVC

```python
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_vec, y_train)

y_pred_knn = knn.predict(X_test_vec)

print("K-NN Accuracy:", accuracy_score(y_test, y_pred_knn))
print(classification_report(y_test, y_pred_knn))
```

```
K-NN Accuracy: 0.8339156005730515
              precision    recall  f1-score   support

           0       0.79      0.02      0.04      6697
           1       0.83      1.00      0.91     33090

    accuracy                           0.83     39787
   macro avg       0.81      0.51      0.47     39787
weighted avg       0.83      0.83      0.76     39787
```

K-NN

```python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# Initialize the model
lr = LogisticRegression(max_iter=1000, random_state=42)

# Train the model
lr.fit(X_train_vec, y_train)

# Predict on the test set
y_pred_lr = lr.predict(X_test_vec)

# Evaluate the model
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr))
```

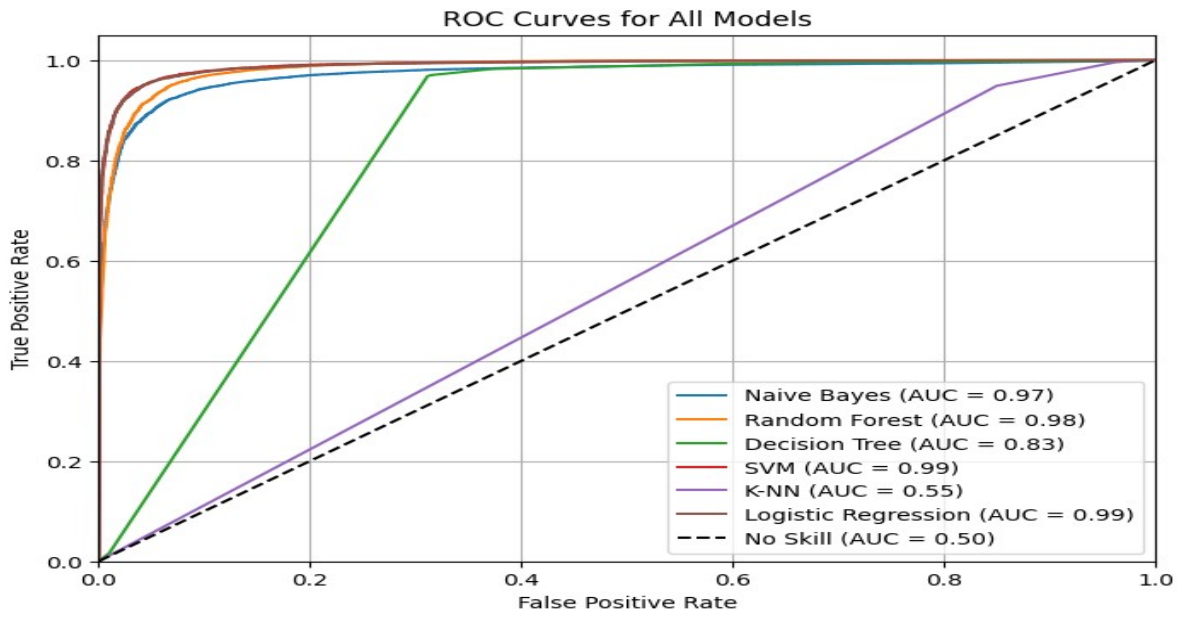
```
Logistic Regression Accuracy: 0.9635810691934551
              precision    recall  f1-score   support

           0       0.91      0.87      0.89      6697
           1       0.97      0.98      0.98     33090

    accuracy                           0.96     39787
   macro avg       0.94      0.93      0.93     39787
weighted avg       0.96      0.96      0.96     39787
```

Logistic Regression

LSTM

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout

model = Sequential()
model.add(Embedding(input_dim=20000, output_dim=128, input_length=200))
model.add(LSTM(64, return_sequences=False))
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

history = model.fit(X_train_pad, y_train, epochs=5, batch_size=64, validation_data=(X_test_pad, y_test))

loss, accuracy = model.evaluate(X_test_pad, y_test)
print("Test Accuracy:", accuracy)
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/embedding.py:90: UserWarning: Argument `input_length` is de
  warnings.warn(
Epoch 1/5
2487/2487 ━━━━━━━━━━━━━━━━━━━━ 42s 14ms/step - accuracy: 0.9544 - loss: 0.1329 - val_accuracy: 0.9874 - val_loss: 0.0401
Epoch 2/5
2487/2487 ━━━━━━━━━━━━━━━━━━━━ 33s 13ms/step - accuracy: 0.9887 - loss: 0.0343 - val_accuracy: 0.9877 - val_loss: 0.0354
Epoch 3/5
2487/2487 ━━━━━━━━━━━━━━━━━━━━ 42s 14ms/step - accuracy: 0.9934 - loss: 0.0195 - val_accuracy: 0.9892 - val_loss: 0.0482
Epoch 4/5
2487/2487 ━━━━━━━━━━━━━━━━━━━━ 40s 13ms/step - accuracy: 0.9970 - loss: 0.0093 - val_accuracy: 0.9900 - val_loss: 0.0348
Epoch 5/5
2487/2487 ━━━━━━━━━━━━━━━━━━━━ 43s 14ms/step - accuracy: 0.9980 - loss: 0.0065 - val_accuracy: 0.9905 - val_loss: 0.0430
1244/1244 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.9905 - loss: 0.0420
Test Accuracy: 0.9904994368553162
```



10. Conclusion

This project successfully demonstrates the application of both traditional and deep learning techniques to the task of **AI-generated vs Human-generated text classification**. With the rise of advanced language models like GPT-3, GPT-J, and ChatGPT, the ability to accurately identify the origin of a text has become increasingly important in fields such as education, journalism, cybersecurity, and ethical AI governance.

Through systematic preprocessing, feature extraction, and model comparison, the following conclusions were drawn:

- **Traditional machine learning models**, particularly **Logistic Regression** and **SVM**, offer strong baseline performance with simple TF-IDF features.
- The **LSTM deep learning model** significantly outperformed all classical models in terms of accuracy, recall, and ROC-AUC, thanks to its ability to capture complex sequential and contextual patterns.
- The final system can serve as a robust detection tool for practical use cases involving content moderation or authorship verification.

11.References

1. **Official documentation for text vectorization using TF-IDF — learning vocabulary and weighting features.**
   (https://scikit-learn.org/stable/modules/generatedsklearn.feature_extraction.text.TfidfVectorizer.html)

2. **Guide explaining how `TfidfVectorizer` transforms text into numeric features for classification.**
   (https://www.slingacademy.com/article/using-tfidfvectorizer-for-text-classification-in-scikit-learn/)
3. Official API documentation for TensorFlow's LSTM layer, explaining its structure and usage.
   (https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM)
4. Research paper describing the TensorFlow architecture and its applicability to deep learning.
   (https://arxiv.org/abs/1605.08695)